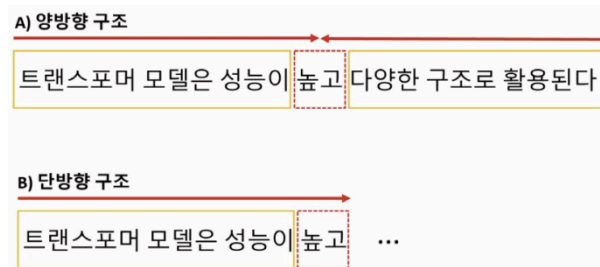




07 트랜스포머

- **트랜스포머(Transformer)** : 2017년 코넬 대학의 아시시 바스와니 등의 연구 그룹이 소개한 신경망 아키텍처
- 순차적 방식 X, 병렬로 입력 시퀀스를 처리하는 기능
 - 긴 시퀀스 - 트랜스포머 모델을 순환 신경망 모델보다 더 빠르고 효율적
- 순차 처리(Sequential Processing), 반복 연결(Recurrent Connections)에 의존 X
- **셀프 어텐션(Self-Attention)** : 입력 토큰 간의 관계를 직접 처리, 이해
 - 재귀 or 합성곱 연산 없이 토큰 간의 관계를 직접 모델링
- 대용량 데이터셋(기계 번역) 작업에 적합
- 언어 모델링 및 텍스트 분류에서 매우 효과적
- 광범위한 자연어 처리 작업에서 높은 효율
- **오토 인코딩(Auto-Encoding)**
 - 랜덤하게 문장의 일부를 빈칸 토큰으로 만들고 해당 빈칸에 어떤 단어가 적절할지 예측하는 작업
 - **인코더(Encoder)** : 예측되는 토큰의 양옆에 있는 토큰들을 참조 - 양방향 구조
- **자기 회귀(Auto-Regressive)**

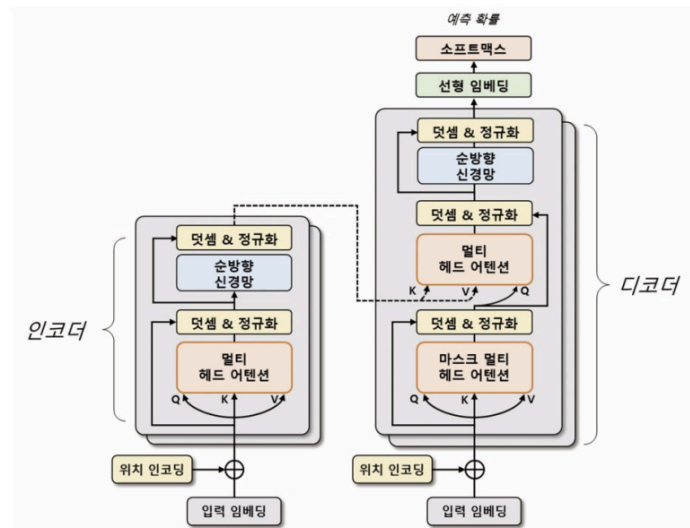
- 단방향성 디코더를 사용
- **디코더(Decoder)** : 예측되는 토큰의 왼쪽에 있는 토큰들만 참조 - 단방향 구조



모델	학습구조	학습방법	학습 방향성
BERT	인코더	오토 인코딩	양방향
GPT	디코더	자기 회귀	단방향
BART	인코더+디코더	오토 인코딩+자기 회귀	양방향+단방향
ELECTRA	인코더+판별기	오토 인코딩+대체 토큰 탐지	양방향
T5	인코더+디코더	오토 인코딩+자기 회귀+다양한 자연어 처리 작업을 학습	양방향

Transformer

- 다양한 자연어 처리 분야(기계 번역, 챗봇, 음성 인식 등)에서 많은 성과를 내는 딥러닝 모델
 - **어텐션 메커니즘(Attention Mechanism)** 사용하여 시퀀스 임베딩 표현
 - 인코더와 디코더 간의 상호작용
 - 입력 시퀀스의 중요한 부분에 초점 맞추어 문맥 이해, 출력 생성
 - 인코더와 디코더 단어 사이의 상관 관계 계산
- 입력 시퀀스의 각 단어가 출력 시퀀스의 어떤 연관 있는지 파악
- 번역 or 요약문 생성과 관련된 작업 수행
- 기존 순환 신경망 모델보다 학습 속도 빠르고, 병렬 처리 가능
 - 대규모 데이터셋에서 높은 성능
 - 임베딩 과정에서 문장 전체 정보를 고려
- 문장 길이가 길어지더라도 성능이 유지!



[트랜스포머 모델 구조]

- 인코더와 디코더는 두 부분으로 구성
- 각각 N개의 **트랜스포머 블록(Transformer Block)**으로 구성
- 블록 - **멀티 헤드 어텐션(Multi-Head Attention)** & **순방향 신경망**으로 구성
 - 멀티 헤드 어텐션 : 입력 시퀀스에서 **쿼리(Query)**, **키(Key)**, **값(Value)** 벡터를 정의 - 입력 시퀀스들의 관계를 셀프 어텐션하는 벡터 표현 방법
 - 쿼리 & 각 키의 유사도를 계산 - 해당 유사도를 가중치로 사용 - 값 벡터를 합산
- 어텐션 행렬 - 입력 시퀀스 각 단어의 임베딩 벡터 대체
- 입력 시퀀스의 단어 사이의 상호작용을 고려 - 임베딩 벡터 갱신
- 순환 신경망 - 임베딩 벡터 고도화를 위해 사용
 - 여러 개의 선형 계층으로 구성
 - 입력 벡터에 가중치를 곱하고, 편향을 더하며, 활성화 함수를 적용
 - 학습된 가중치들은 입력 시퀀스의 각 단어의 의미를 잘 파악할 수 있는 방향으로 갱신
- 입력 시퀀스 데이터를 **소스(Source)** & **타겟(Target)** 데이터로 나누어 처리
- 인코더 - 소스 시퀀스 데이터를 **위치 인코딩(Positional Encoding)**된 입력 임베딩으로 표현 - 트랜스포머 블록의 출력 벡터 생성
 - 출력 벡터 : 입력 시퀀스 데이터의 관계를 잘 표현할 수 있게 구성됨
- 디코더 - 트랜스포머 블록으로 구성, **마스크 멀티 헤드 어텐션(Masked Multi-Head Attention)** 사용 → 타겟 시퀀스 데이터를 순차적으로 생성
 - 인코더의 출력 벡터 정보 참조 ← 디코더 입력 시퀀스들의 관계를 고도화하기 위해
- 최종 생성된 디코더 출력 벡터 : 선형 임베딩으로 재표현 - 이미지 or 자연어 모델에 활용

입력 임베딩과 위치 인코딩

- 입력 시퀀스의 각 단어 : 임베딩 처리되어 벡터 형태로 변환
- 트랜스포머 모델 - 입력 시퀀스를 병렬 구조로 처리 → 단어의 순서 정보를 제공 X

⇒ 위치 정보를 임베딩 벡터에 추가해 단어의 순서 정보를 모델에 반영해야 함

⇒ 위치 인코딩 방식 사용

[위치 인코딩]

- 입력 시퀀스의 순서 정보를 모델에 전달하는 방법
- 각 단어의 위치 정보를 나타내는 벡터를 더해서 임베딩 벡터에 위치 정보를 반영
- Sin 함수와 cos 함수를 사용해 생성
- 임베딩 벡터와 위치 정보가 결합된 최종 입력 벡터를 생성

→ 단어의 순서 정보를 학습 가능

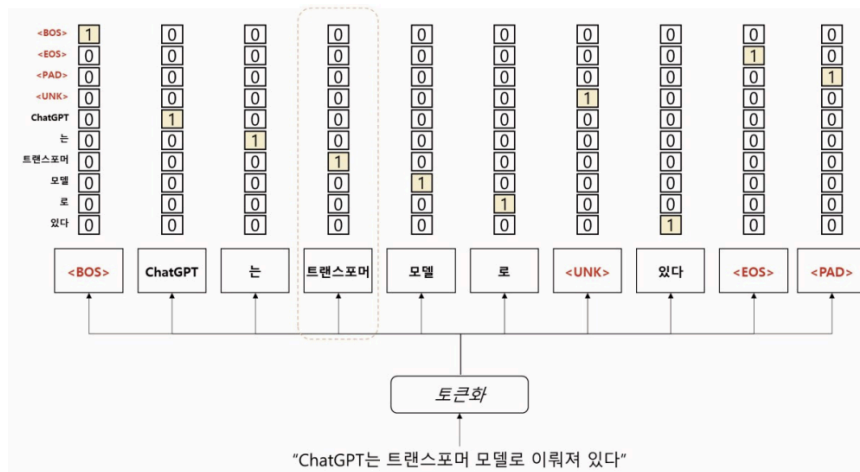
→ 토큰의 위치마다 동일한 임베딩 벡터를 사용하지 X - 각 토큰의 위치 정보를 학습 가능

→ 입력 시퀀스의 순서 정보 보존을 위한 중요한 방법 중 하나

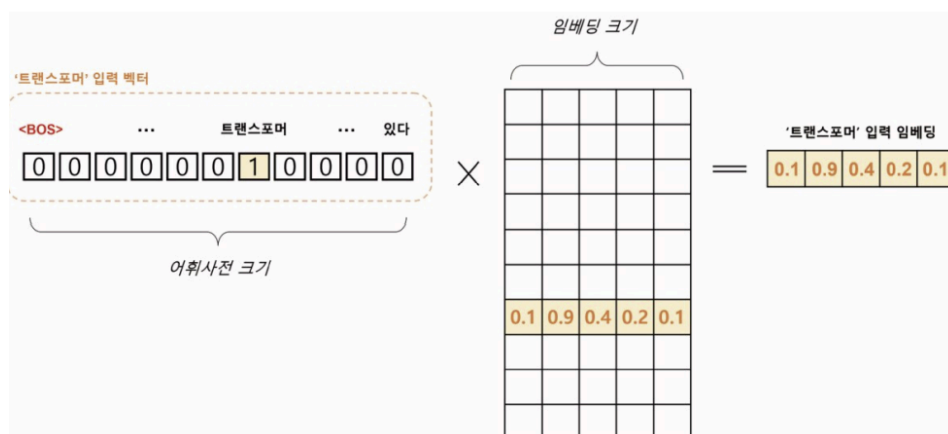
$$PE(pos, 2i) = \sin(pos/10000^{2i/d_{model}})$$
$$PE(pos, 2i+1) = \cos(pos/10000^{2i/d_{model}})$$

특수 토큰

- 특수 토큰을 활용하여 문장 표현 가능
- 입력 시퀀스의 시작과 끝을 나타내거나 **마스킹(Masking)** 영역으로 사용
 - 마스킹 - 일부 입력을 무시 가능



- BOS(Beginning of Sentence), EOS(End of Sentence), UNK(Unknown), PAD(Padding) 토큰 등
 - BOS 토큰 : 문장의 시작
 - EOS 토큰 : 문장의 종료
 - UNK 토큰 : 모르는 단어(이전에 본 적 없는)
 - PAD 토큰 : 모든 문장을 일정한 길이로 맞추기 위해 사용(짧은 문장의 빈 공간 채우기)
- 생성된 문장 토큰 배열을 어휘 사전에 등장하는 위치에 원-핫 인코딩으로 표현



[원-핫 벡터(트랜스포머) → 입력 임베딩으로 변환되는 과정]

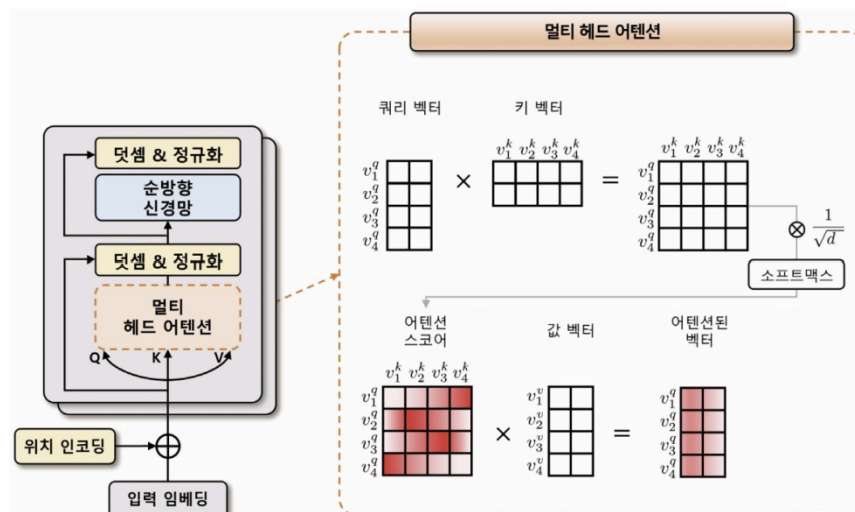
- 어휘 사전의 크기 : V
- 입력 임베딩 차원 : d
- 임베딩 행렬 $[V, d]$ 에 의해 $[1, d]$ 크기의 벡터로 변환

→ N개의 문장이 최대 S개의 토큰 길이를 가질때 $[N, S, V]$ 크기의 원-핫 벡터 텐서 → $[N, S, d]$ 크기의 임베딩 텐서로 변환

- 임베딩 텐서 : 입력으로 사용 / 트랜스포머의 모든 계층에서 공유되는 텐서로 사용

트랜스포머 인코더

- 입력 시퀀스를 받아 여러 개의 계층으로 구성된 인코더 계층을 거쳐 연산 수행
- 각 인코더 계층 : 멀티 헤드 어텐션 & 순방향 신경망으로 구성
- 입력 데이터에 대한 정보 추출 → 다음 계층으로 전달
- 임베딩 벡터를 입력 벡터에 더함(← 위치 정보를 반영하기 위해)



[트랜스포머 인코더 블록의 연산 과정]

- 위치 인코딩이 적용된 소스 데이터의 입력 임베딩을 입력받음
- 입력 텐서 차원 : $[N, S, d] \rightarrow$ 입력 임베딩은 선형 변환을 통해 3개의 임베딩 벡터를 생성
- 생성된 3개의 벡터 : 쿼리(Q), 키(K), 값(V) 벡터라고 정의
 - 쿼리 벡터
 - 현재 시점에서 참조하고자 하는 정보의 위치를 나타내는 벡터
 - 인코더의 각 시점마다 생성
 - 현재 시점에서 질문이 되는 벡터
 - 이 벡터 기준으로 다른 시점의 정보를 참조

- 키 벡터

- 쿼리 벡터와 비교되는 대상
- 쿼리 벡터를 제외한 입력 시퀀스에서 탐색되는 벡터
- 인코더의 각 시점에서 생성

- 값 벡터

- 쿼리 벡터와 키 벡터로 생성된 어텐션 스코어를 얼마나 반영할지 설정하는 가중치 역할

[어텐션 스코어 계산식]

$$score(v^q, v^k) = \text{softmax}\left(\frac{(v^q)^T \cdot v^k}{\sqrt{d}}\right)$$

[셀프 어텐션 과정]

- 쿼리, 키, 값 벡터를 내적해 어텐션 스코어를 구하고, 이 스코어값에 루트 d 만큼 나눠 보정
 - 보정값은 벡터 차원이 커질 때 스코어값이 같이 커지는 문제를 완화하기 위해 적용
- 보정된 어텐션 스코어를 소프트맥스 함수를 이용하여 확률적으로 재표현
- 값 벡터와 내적하여 셀프 어텐션된 벡터를 생성

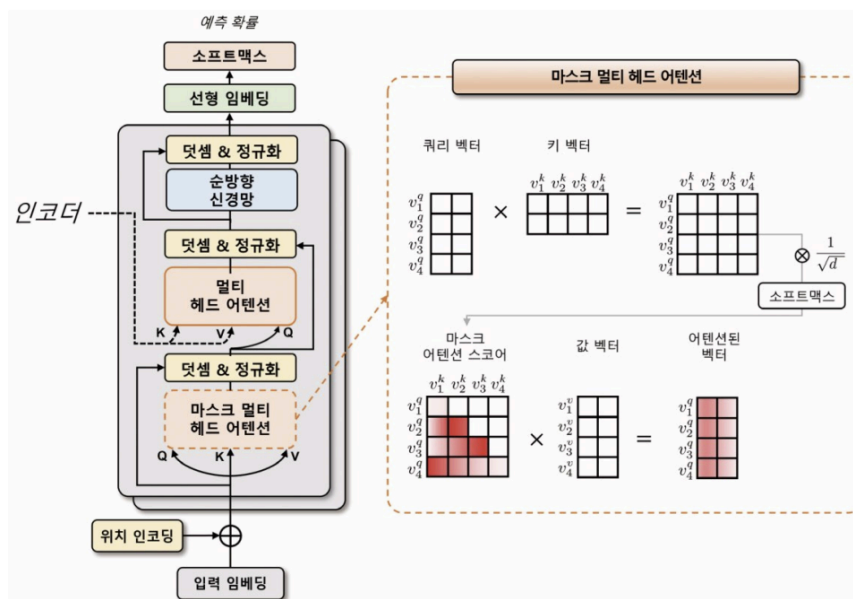
→ 위 과정 반복 / 셀프 어텐션된 스코어 맵 생성

- **멀티 헤드(Multi-Head)** : 셀프 어텐션을 여러 번 수행해 여러 개의 헤드를 만들
 - 각각의 헤드가 독립적으로 어텐션을 수행, 결과를 합침
- 생성된 k개의 셀프 어텐션 벡터는 임베딩 차원 축으로 다시 **병합(concatenation)**되어 [N, S, d] 형태로 출력됨
- 덧셈 & 정규화는 멀티 헤드 어텐션을 통과하기 이전의 입력값 [N, S, d] 텐서와 통과한 이후의 출력값 [N, S, d] 텐서를 더함으로써 학습 시 발생하는 기울기 소실을 완화
 - 이 값에 임베딩 차원 축으로 정규화하는 계층 정규화를 적용
- 순방향 신경망은 두 가지 방법 적용 가능
 - 선형 임베딩
 - ReLU 로 이뤄진 인공 신경망 or 1차원 합성곱
- 이어서 덧셈 & 정규화 과정 수행
- 인코더 : 여러 개의 트랜스포머 인코더 블록으로 구성됨
- 이전 블록에서 출력된 벡터 - 다음 블록의 입력으로 전달되어 인코더 블록을 통과 → 입력 시퀀스의 정보 추상화

- 마지막 인코더 블록에서 출력된 벡터는 디코더에서 사용
- 디코더의 멀티 헤드 어텐션 모듈에서 참조되는 키, 값, 벡터로 활용
- → 인코더와 디코더가 서로 정보 공유

트랜스포머 디코더

- 위치 인코딩이 적용된 타깃 데이터의 입력 임베딩을 입력받음
 - 입력 시퀀스 내에서 각 단어의 상대적인 위치 정보를 전달하는 기법
 - 디코더의 입력 임베에 위치 정보 추가 → 입력 시퀀스의 순서 정보 학습 가능
 - 인코더의 멀티 헤드 어텐션 모듈 - **마스크 멀티 헤드 어텐션 모듈**로 대체(**인과성(Casuality)** 반영)
 - 마스크 멀티 헤드 어텐션 모듈
 - 첫 번째 쿼리 벡터가 첫 번째 키 벡터만을 바라보도록 마스크를 씌움
 - 두 번째 쿼리 벡터는 첫 번째와 두 번째 키 벡터를 바라보게 마스크를 씌움
- 셀프 어텐션에서 현재 위치 이전의 단어들만 참조 가능
- 인과성이 보장
- 마스크 영역에 수치적으로 작은 값인 -inf 마스크를 더해줌 → 해당 영역의 어텐션 스코어 값이 0에 가까워짐
 - 소프트맥스 계산 시 어텐션 가중치 : 0 → 마스크 영역 이전의 입력 토큰들에 대한 정보 참조 X



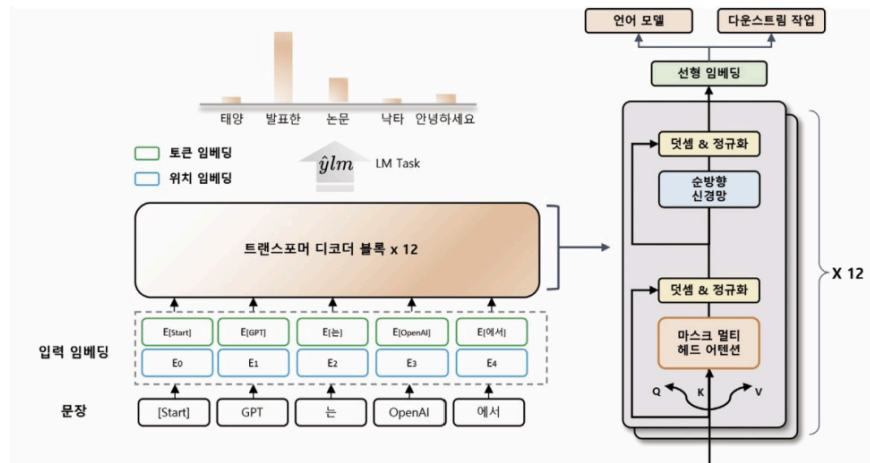
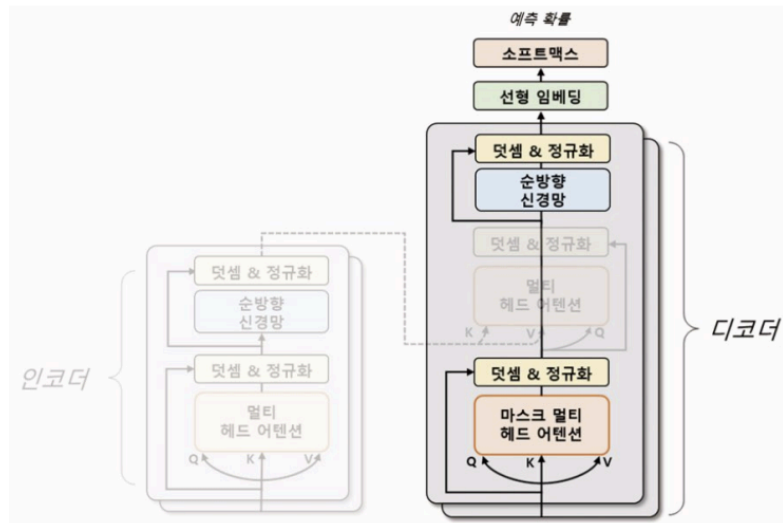
추론 순서	디코더 입력	디코더 출력
1	[BOS], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
2	[BOS], 'ChatGPT', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]
3	[BOS], 'ChatGPT', '는', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD], [PAD]
4	[BOS], 'ChatGPT', '는', '트랜스포머', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD]
5	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD], [PAD]
6	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD], [PAD], [PAD]
7	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD], [PAD]	[BOS], 'ChatGPT', '는', '트랜스포머', '모델로', '이뤄져', '있다', [EOS], [PAD], [PAD], [PAD]

GPT(Generative Pre-trained Transformer)

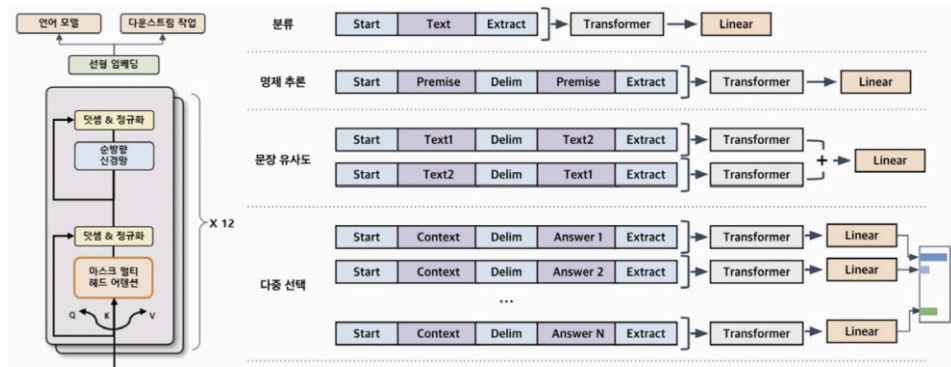
- 2018년 OpenAI에서 발표한 트랜스포머 기반 언어 모델로 가장 널리 알려진 언어 모델
- ELMo(Embeddings from Language Model)과 같이 대규모 말뭉치로 사전 학습된 모델 - 다운스트림 작업에서 우수한 성능
- 트랜스포머의 디코더를 여러 층 쌓아 만든 언어 모델
- 인코더 - 입력 문장의 특징 추출 / 디코더 - 입력 문장과 출력 문장 간의 관계를 이해하고 출력 문장 생성에 초점
- 자연어 생성 등의 언어 모델링 작업에서 높은 성능
- 기계 번역, 질의응답, 요약 등의 자연어 처리 작업에 활용
- 많은 양의 매개변수를 줄이기 위해 **RLHF(Reinforcement Learning from Human Feedback)** 방법 도입

GPT-1

- 트랜스포머 구조를 바탕으로 한 **단방향(Undirectional)** 언어 모델
 - 텍스트 생성 시 이전 단어를 차례로 입력 → 다음 단어를 예측
 - 모델의 계산량 감소 - 학습 속도 증가, 모델 크기 감소
- 이전 단어들에 대한 정보로 다음 단어를 예측할 때 트랜스포머 구조를 활용
- 트랜스포머 구조 중 디코더 부분만 사용, 인코더-디코더 간 어텐션 연산 제외



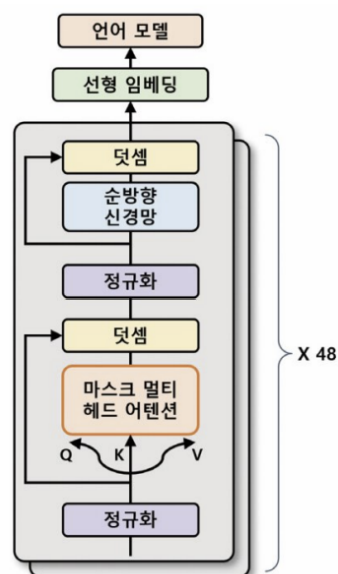
- 미세 조정을 위한 다운스트림 작업
 - 입출력 구조 바꾸지 X, 언어 모델을 **보조 학습(Auxiliary Learning)**해 사용
 - 보조 학습 : 주요 학습 작업 외에 다른 작은 부가적인 학습 작업을 동시에 수행하는 것
 - 학습한 정보를 주요 작업에 전달하여 성능 향상 가능



- 일반적인 손실 함수 외에 추가적인 손실 함수 필요
 - 다운스트림 작업의 목표를 달성하기 위해 필요한 정보 학습 가능
- 특수 토큰 활용
 - <start>, <delim>, <extract>

GPT-2

- 1024 길이까지 입력 처리 가능
- 48개의 디코더 계층을 사용해 GPT-1보다 복잡한 패턴 학습 가능
- 15억개의 매개변수 사용 - 언어 모델링 작업 수행



- GPT-1보다 훨씬 많은 양의 데이터 이용해 사전 학습
- 자연스러운 문장 생성 능력 - 다양한 분야에서 활용
- 제로-샷 학습 가능
- 별도 미세 조정 X 다운스트림 작업에서 사용 가능 ← 사전 학습 과정에서 특정한 형식의 데이터를 입력

GPT-3

- GPT-2와 거의 동일한 모델 구조 보유
- But, GPT-2보다 약 116배 많은 매개변수 보유
- 매우 규모가 큰 모델
 - 96개의 헤드를 가진 멀티 헤드 어텐션 사용
 - 트랜스포머 디코더 층 : 96개 사용
- **희소 어텐션**, 일반 어텐션 섞어서 사용 ← 멀티 헤드 어텐션 과정에서 연산량 줄이기 위함
- 토큰 임베딩 크기 1600 → 12888
- 최대 2048개까지 토큰 입력 가능 - 더욱 긴 문장에 대한 처리 능력 향상
- **프롬프트(prompt)** 형식으로 작업 수행

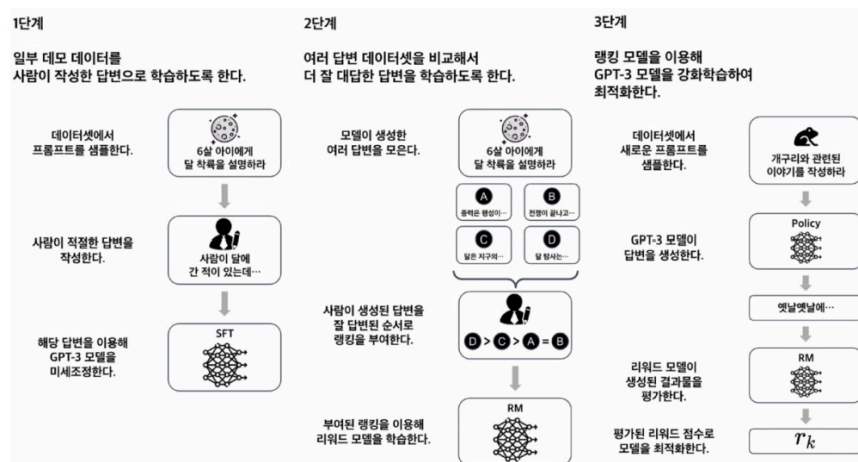
질의 응답 입력 프롬프트 문서 : 생성적 사전학습 트랜스포머(Generative Pre-Trained Transformer 3), GPT-3는 OpenAI에서 만든 대형 언어 모델이다. 질문 : GPT-3를 만든 곳은 어디인가? 답변 : OpenAI 출력값	자연어 추론 입력 프롬프트 자연어 추론 : OpenAI는 미국의 인공지능 연구소로, 인류에게 이익을 주는 것을 목표로 한다. GPT-3등 거대 언어 모델을 개발하여 많은 이목을 끌었다. 질문 : OpenAI는 상장사인가? 참, 거짓, 둘 다 아님 답변 : 둘 다 아님 출력값
수학 문제 풀이 입력 프롬프트 질문 : $(2 * 4) * 6$은 무엇인가? 답변 : 48 출력값	일반 상식 질의 응답 입력 프롬프트 질문 : 물은 섭씨 몇 도에서 어는가? 답변 : 0도 출력값

- 새로운 도메인 작업에 대해서도 높은 일반화 능력 발휘
- 자연어 처리 작업에 대한 일반화 능력 상승
- But, 인간적인 이해력과 상식적인 추론 능력 부족

- 결과물 사용 시 검토 & 검증하는 과정 필요
- 생성된 텍스트의 저작권 - 학습시킨 데이터와 모델을 만든 회사가 소유

GPT-3.5(InstructGPT)

- GPT-3의 모델 구조 따르면서, 새로운 학습 방법인 RLHF 도입
 - 매개변수 수 감소, 모델의 자연스러움 증가



- RLHF 학습 방법
 - 데이터셋에서 임의의 프롬프트 가져옴
 - 프롬프트에 대해 사람이 직접 답을 작성하게 함
 - 답변 : 모델이 생성한 문장과 함께 평가 - 자연스러움 → 보상
 - 답변 : 모델이 생성한 문장과 함께 평가 - 부자연스러움 → 보상 X
 - 보상을 기반으로 강화학습을 통해 모델 학습
 - 지도 미세 조정(Supervised Fine-Tuning, SFT)
- 학습 데이터양 고려 - 전체 프롬프트에 사람이 답변 다는 것 - 어려움
- GPT 모델이 얼마나 잘 답변했는지 평가하는 보상 모델 학습
- 시드(seed)에 따라 다른 답변 생성 - 내용이 서로 다른 여러 개의 답변 생성 가능
- 주어진 프롬프트(State)에 대해 GPT-3 언어 모델(Policy)가 답변을 생성(Action), 사람의 피드백을 기반으로 한 학습된 보상 모델을 평가(Reward)

GPT-4

- 이전 모델에 비해 향상된 성능
- But, 일부 입력에 대해 현실적이지 않은 결과를 생성하는 **환각(Hallucination) 현상**이 아직 존재
 - 모델이 인간과 같은 추론과 판단 결과를 갖추지 못하고 부적절하거나 오류가 많은 답변을 제시하는 현상
- 텍스트 데이터뿐만 아니라 이미지 데이터도 인식 가능한 **멀티모달(Multimodal) 모델**
- 최대 32768개의 토큰 입력 가능
- 25000개의 단어 처리 가능
- ChatGPT Plus나 마이크로소프트 Bing을 통해 모델 사용 가능